VOLUME 12

COMPLETE VOLUME — CHAPTERS 12.1–12.10

AI Development Handbook

The Final Volume — How Any AI Session Implements This Project

Document Title	Volume 12 — AI Development Handbook (Combined)
Volume Reference	Volume 12
Chapters Included	12.1 through 12.10 (10 chapters)
Version	1.0
Status	Draft — Pending Approval
Owner	Faisal — Human Archive
Prepared With	Claude (Cowork)
Date	05 August 2026
Governed By	Volume 0 — Project Foundation; operationalizes Volumes 3, 6, 7, 9, 11

Table of Contents

This volume is the last foundation document before implementation begins. It fixes how any AI session collaborates on this codebase: collaboration rules, prompt templates, code generation and documentation standards, three review checklists, refactoring rules, git workflow, and the single Definition of Done every story is held to. Once this volume is approved, the 13-step implementation sequence (Volume 11, Chapter 11.1) may begin.

VOLUME 12 — AI DEVELOPMENT HANDBOOK

CHAPTER 12.1

AI Collaboration Rules

How Any AI Session Picks Up This Project With Full Context

Document Title	AI Collaboration Rules
Chapter Reference	12.1
Version	1.0
Status	Draft — Pending Approval
Owner	Faisal — Human Archive
Prepared With	Claude (Cowork)
Date	05 August 2026
Governed By	Volume 0, Chapter 0.2 — the Constitution this entire volume operationalizes.

1. Purpose

Volumes 0–11 exist so a person or an AI collaborator can implement this app without re-deriving decisions already made. This chapter is the entry point: the rules an AI session follows before touching any code, so Volumes 1–11 are actually used rather than silently re-invented.

2. The Non-Negotiable Rule

Read before you write: before generating any code, an AI session locates and reads the specific Volume/Chapter that governs the area it's about to touch. If no chapter covers it, that itself is a signal to stop and flag a documentation gap (Chapter 11.8) rather than improvise a new pattern.

3. Session Startup Checklist

- Confirm which Roadmap step (Volume 11, Chapter 11.1) the current task belongs to, and which Milestone (Chapter 11.2) it feeds.
- Check the Feature Tracker (Chapter 11.6) for the FR group's current status before assuming a clean starting point.
- Check the Bug Tracker (Chapter 11.7) for open bugs in the area about to be touched.
- Confirm the relevant ADR (Volume 3/4/6/8) if the task touches a decision already made — never silently substitute a different library or pattern than what an ADR fixed.

4. Escalation, Not Improvisation

When a task requires a decision no volume has made (a genuinely new architectural choice, not a re-derivation of an existing one), the AI session stops and surfaces the question to Faisal rather than deciding unilaterally — mirroring Volume 0's human-approval gate at the code level.

5. Scope Discipline

An AI session works only within the current Roadmap step (Volume 11, Chapter 11.1) unless explicitly asked to range wider. Touching Phase 2+ scope (Volume 1, Chapter 1.11) without being asked is scope creep even if the AI 'noticed something useful to add.'

6. What the Rest of This Volume Covers

- Prompt Templates (12.2) — how to ask an AI session to do a specific kind of task consistently.
- Code Generation Rules (12.3), Documentation Rules (12.4) — the standards generated output is held to.

- Review Checklists (12.5, 12.6, 12.7) — how output is checked before it's accepted.
- Refactoring Rules (12.8), Git Workflow (12.9), Definition of Done (12.10) — how change is made and called complete.

VOLUME 12 — AI DEVELOPMENT HANDBOOK

CHAPTER 12.2

Prompt Templates

Consistent Ways to Ask for Common Kinds of Work

Document Title	Prompt Templates
Chapter Reference	12.2
Version	1.0
Status	Draft — Pending Approval
Owner	Faisal — Human Archive
Prepared With	Claude (Cowork)
Date	05 August 2026

1. Why Templates

A consistent prompt shape means a consistent quality of output — each template below names the exact volume/chapter context an AI session needs before it can do the task correctly, so context isn't left to chance.

2. Template — Implement a Feature

```
Implement <FR-XXX-NN> from Volume 1, Chapter 1.3.
Architecture: Volume 3/6 (layer + module it belongs in).
Related ADRs: <list, if any>.
Acceptance: Volume 1 US-XX's acceptance criteria + Chapter 12.10 DoD.
Update: Feature Tracker (11.6), Changelog (11.5) on completion.
```

3. Template — Fix a Bug

```
Fix BUG-XXX (Volume 11, Chapter 11.7).
Severity: <S1-S4>.
Root cause hypothesis: <if known>.
Add a regression test (Volume 9, Ch.9.6/9.7) covering this case.
Update: Bug Tracker status, Changelog 'Fixed' entry.
```

4. Template — Refactor

```
Refactor <area> per Chapter 12.8's rules.
Reason: <readability / duplication / boundary violation>.
Constraint: no behavior change — existing tests (Volume 9) must still pass unmodified.
Constraint: does not cross a layer/module boundary fixed in Volume 3, Ch.3.4/3.5.
```

5. Template — Architecture Question

```
Question: <decision needed>.
Checked: Volume 3/4/6/8 ADRs — none cover this.
Options: <A, B, C with tradeoffs>.
Do not decide — escalate per Chapter 12.1, Section 4.
```

6. General Rule

Every prompt names its governing chapter(s): a prompt that doesn't reference which Volume/Chapter it implements against is treated as incomplete — the AI session should ask for that reference before starting, per Chapter 12.1's read-before-you-write rule.

VOLUME 12 — AI DEVELOPMENT HANDBOOK

CHAPTER 12.3

Code Generation Rules

The Standard Any AI-Generated Code Is Held To

Document Title	Code Generation Rules
Chapter Reference	12.3
Version	1.0
Status	Draft — Pending Approval
Owner	Faisal — Human Archive
Prepared With	Claude (Cowork)
Date	05 August 2026

1. Baseline

AI-generated code is never held to a lower bar than human-written code — it follows Volume 3, Chapter 3.7's coding standards (Effective Dart, analysis_options.yaml, import-boundary lints) exactly, with zero exceptions for 'the AI wrote it.'

2. Rules

- Never invent a dependency not already in Volume 3, Chapter 3.1's tech stack or Chapter 3.8's dependency policy — a genuinely new package need is an architecture question (Chapter 12.2, Section 5), not a silent addition.
- Never bypass a layer boundary (Volume 3, Ch.3.4) to 'make it simpler' — a widget importing Drift directly is a defect, not a shortcut, even if it works.

Generated code is placed, not scattered: Volume 3, Chapter 3.6's folder structure is authoritative — a new file goes exactly where that structure says its layer/module belongs, never wherever is most convenient in the moment.

- *.g.dart files (Chapter 3.1's freezed/json_serializable/riverpod_generator) are generated via build_runner, never hand-written or hand-edited — matching Volume 7, Chapter 7.12's drift-check discipline.
- print() is never used for diagnostics — Volume 3, Chapter 3.7's structured logger is the only sanctioned output, so Volume 9's tooling and Volume 10's monitoring can filter it.
- A generated function/class touching domain/ or data/ carries a dartdoc comment (Chapter 3.7's public_member_api_docs rule) — this is not optional for AI-generated code.
- Traceability comments referencing the FR/BR/US/ADR ID being implemented are added at the point of implementation, not left for a reviewer to reverse-engineer (extends Chapter 3.7's rule into generation itself).

3. Security-Sensitive Generation

Code touching auth tokens, chunk encryption assumptions (Volume 8, ADR-011), or PII fields (Volume 8, Chapter 8.2) is generated conservatively — when in doubt, the AI session asks rather than assumes a permissive default, since Volume 8's threat model treats over-permissive defaults as a first-class risk.

4. Self-Check Before Presenting Output

- Does this match an existing pattern elsewhere in the codebase, or does it quietly introduce a second way of doing the same thing?
- Would this pass Chapter 12.5's Code Review Checklist unmodified?

VOLUME 12 — AI DEVELOPMENT HANDBOOK

CHAPTER 12.4

Documentation Rules

Keeping Volumes 0–11 True Once Implementation Starts

Document Title	Documentation Rules
Chapter Reference	12.4
Version	1.0
Status	Draft — Pending Approval
Owner	Faisal — Human Archive
Prepared With	Claude (Cowork)
Date	05 August 2026

1. The Rule

A code change that alters behavior described in Volumes 1–11 updates the affected chapter in the same change — not as a follow-up task, not 'later.' This is the single rule Volume 11, Chapter 11.8 references as enforced specifically for AI-assisted work.

2. What Counts as 'Alters Behavior'

- An FR/BR/US is implemented differently than its chapter describes (even if arguably better) — the chapter is updated to match reality, with a note on why it changed.
- An ADR's decision is revisited (e.g. swapping Dio for another HTTP client) — this requires a new ADR entry (superseding, not silently deleting, the old one), not just a code change.
- A new dependency is added — Volume 3, Chapter 3.8's table is updated in the same change.

3. What Does Not Require a Doc Update

- Internal implementation detail with no externally-observable behavior change (e.g. renaming a private variable).
- A bug fix that restores documented behavior — the Bug Tracker (Volume 11, Ch.11.7) and Changelog (Ch.11.5) are the record; no volume chapter needs editing if the fix simply matches what was already specified.

4. Traceability Comments in Code

Every non-trivial function implementing a specific FR/BR/US carries a short comment naming the ID (Volume 3, Ch.3.7) — this is the cheapest link between code and documentation, and it is checked in code review (Chapter 12.5).

5. Ownership of Doc Updates

Whoever's change caused the drift writes the doc update — an AI session generating the code also generates the accompanying chapter edit in the same turn, per Chapter 12.1's read-before-you-write rule applied in reverse: write, then reconcile.

6. Review Gate

Code review (Chapter 12.5) explicitly checks 'does this change require a Volume 1-11 update, and if so, is it included' — a change is not approved with an acknowledged doc gap left open.

VOLUME 12 — AI DEVELOPMENT HANDBOOK

CHAPTER 12.5

Code Review Checklist

Extends Volume 3, Chapter 3.7's Checklist Into a Full Gate

Document Title	Code Review Checklist
Chapter Reference	12.5
Version	1.0
Status	Draft — Pending Approval
Owner	Faisal — Human Archive
Prepared With	Claude (Cowork)
Date	05 August 2026

1. Relationship to Volume 3, Chapter 3.7

Chapter 3.7 already lists a short review checklist. This chapter is that same checklist, expanded into the full gate every pull request — human- or AI-authored — passes through before merge, per Volume 7, Chapter 7.6's PR-only rule.

2. Checklist

- Static analysis runs clean, zero suppressed lint warnings (Volume 3, Ch.3.7).
- No layer/module boundary violation (Volume 3, Ch.3.4/3.5) — checked by the custom_lint import rules, and by eye for anything the linter can't catch.
- Traceability: the FR/BR/US/ADR ID this change implements is identifiable, in the commit message (Volume 7, Ch.7.6) or code comment (Ch.12.4).
- Tests exist and pass for the changed behavior (Volume 9, Ch.9.6/9.7) — a change with zero new/updated tests is only acceptable for pure refactors (Chapter 12.8) with existing coverage.
- Generated files (*.g.dart) are committed and match a fresh build_runner run (Volume 7, Ch.7.12's drift check).
- No secret, API key, or service-account credential is committed (Volume 7, Ch.7.6/7.10).
- Documentation is updated if Chapter 12.4's criteria apply — not left as a follow-up.
- For AI-generated code specifically: does it match an existing pattern rather than introducing a redundant second one (Chapter 12.3, Section 4)?

3. Severity of a Failed Check

Any single unchecked item blocks merge — this checklist has no 'minor exceptions' tier. If an item genuinely doesn't apply (e.g. no tests possible for a pure config change), the reviewer notes why, rather than silently skipping it.

4. Reviewer of Last Resort

For a solo/small team (Volume 11, Ch.11.4's bus-factor risk), Faisal is the final reviewer on anything touching Recording/Upload (Volume 5) or security (Volume 8) — an AI session's own self-review (Chapter 12.3, Section 4) is not a substitute for this on core-promise code.

CHAPTER 12.6

Architecture Review Checklist

For Any Change That Touches a Decision Already Made

Document Title	Architecture Review Checklist
Chapter Reference	12.6
Version	1.0
Status	Draft — Pending Approval
Owner	Faisal — Human Archive
Prepared With	Claude (Cowork)
Date	05 August 2026

1. When This Checklist Applies

Chapter 12.5 gates ordinary code changes. This chapter gates anything that touches an ADR (Volume 3/4/6/8), a layer boundary (Volume 3, Ch.3.4), or the module map (Ch.3.5) — a higher bar because these decisions are load-bearing for the whole codebase, not local to one file.

2. Checklist

- Does this change conflict with an existing ADR? If yes, is a new ADR entry being written to formally supersede it (Ch.12.4, Section 2), or is the change actually out of scope?
- Does this introduce a new cross-module dependency not shown in Volume 3, Chapter 3.5's module map? If yes, is the map being updated, or is the dependency avoidable?
- Does this change the deterministic S3 key scheme (Volume 5, Ch.5.14) or the chunking mechanism (Volume 5, Ch.5.3)? These are treated as especially high-risk — any change here is escalated per Chapter 12.1, Section 4 before proceeding.
- Does this affect the multi-cloud seam (Volume 4, Ch.4.9) between AWS and Firebase? If yes, is the boundary still as narrow as Chapter 4.9 describes, or has it grown?
- Is there a simpler option already in the tech stack (Volume 3, Ch.3.1) before reaching for something new?

3. Escalation Path

An architecture-level change is never merged on an AI session's own authority — it requires Faisal's explicit sign-off, following the same pattern as Volume 0's original ADR process (Volume 3, Ch.3.1).

4. Output of a Passed Review

A new or amended ADR entry, plus updates to any Volume 3/4/6/8 chapter whose description the change affects — an architecture change is not complete until the documentation reflects it (Chapter 12.4).

VOLUME 12 — AI DEVELOPMENT HANDBOOK

CHAPTER 12.7

Testing Checklist

The Practical Gate Applying Volume 9's Standards to Every Change

Document Title	Testing Checklist
Chapter Reference	12.7
Version	1.0
Status	Draft — Pending Approval
Owner	Faisal — Human Archive
Prepared With	Claude (Cowork)
Date	05 August 2026

1. Relationship to Volume 9

Volume 9 defines what testing looks like for this project (unit/integration split, coverage targets, device matrix, UAT). This chapter is the checklist an AI session runs through per change, so Volume 9's standards are actually applied rather than referenced and skipped.

2. Per-Change Checklist

- Unit tests added/updated for new domain/data-layer logic (Volume 9, Ch.9.6).
- Integration test added if the change crosses a repository/backend boundary (Ch.9.7).
- Coverage on the changed files meets or exceeds Volume 9, Chapter 9.5's targets — not just 'the suite still passes.'
- If the change touches Recording/Upload (Volume 5), it is verified against Chapter 9.9's device matrix, not just the primary dev device.
- If the change touches a performance-sensitive path (Volume 9, Ch.9.4's targets), the target is re-measured, not assumed unaffected.
- A bug fix (Volume 11, Ch.11.7) includes a regression test reproducing the original defect.

3. What 'Green CI' Actually Means

Volume 7, Chapter 7.13's CI running green is necessary but not sufficient — this checklist's items (especially coverage-on-changed-files and device-matrix checks for Recording/Upload) are not fully automatable and are confirmed manually before Chapter 12.10's Definition of Done is met.

4. Before UAT (Milestone M11)

Volume 9, Chapter 9.8's manual test plan is run in full, not sampled — every item on that plan is either passed or has an open Bug Tracker entry (Volume 11, Ch.11.7) before UAT sign-off is requested.

VOLUME 12 — AI DEVELOPMENT HANDBOOK

CHAPTER 12.8

Refactoring Rules

Changing Structure Without Changing Behavior

Document Title	Refactoring Rules
Chapter Reference	12.8
Version	1.0
Status	Draft — Pending Approval
Owner	Faisal — Human Archive
Prepared With	Claude (Cowork)
Date	05 August 2026

1. Definition

A refactor changes code structure with zero observable behavior change — if a test needs to change to make a 'refactor' pass, it isn't a refactor, it's a behavior change wearing a refactor's name, and it's reviewed under Chapter 12.5's full checklist instead.

2. Rules

Existing tests are the proof, unmodified: Volume 9's existing unit/integration tests (Ch.9.6/9.7) must pass without edits before and after a refactor — if they need editing, the change has crossed into behavior-altering territory.

- A refactor never crosses a layer boundary (Volume 3, Ch.3.4) in the same change as a feature — moving code across boundaries is its own separate, reviewed change.
- A refactor that touches an ADR's implementation detail (without changing the decision itself) still gets a note added to the relevant Volume 3/4/6/8 chapter if the description there becomes stale (Ch.12.4).
- Large refactors are split into small, independently-revertible commits (Ch.12.9) rather than one large diff — smaller diffs are what actually get reviewed carefully.

3. When Not to Refactor

- Mid-feature, unless the refactor is a small precondition the feature genuinely needs — a feature PR that also 'cleans up' unrelated code makes review harder and is split out.
- On code with zero test coverage — a refactor without a safety net is a rewrite in disguise; add characterization tests first (Volume 9, Ch.9.6) if coverage is missing.

4. AI-Specific Guidance

An AI session proposing a refactor states explicitly what motivated it (duplication, boundary violation, readability) per Chapter 12.2's template — 'this seemed cleaner' is not sufficient justification on its own for touching working code.

VOLUME 12 — AI DEVELOPMENT HANDBOOK

CHAPTER 12.9

Git Workflow

Applying Volume 7, Chapter 7.6's Setup as a Daily Practice

Document Title	Git Workflow
Chapter Reference	12.9
Version	1.0
Status	Draft — Pending Approval
Owner	Faisal — Human Archive
Prepared With	Claude (Cowork)
Date	05 August 2026

1. Relationship to Volume 7, Chapter 7.6

Chapter 7.6 set up the repository's branch model, hooks, and .gitignore. This chapter is how that setup is actually used day to day, including by an AI session generating commits.

2. Branching

- main is always deployable (Ch.7.6) — never committed to directly, including by an AI session; every change goes through feature/<short-description>.
- A branch is scoped to one Roadmap step or one Bug Tracker entry (Volume 11, Ch.11.1/11.7) — not a grab-bag of unrelated changes.
- A branch is merged via pull request and passes Chapter 12.5's (or 12.6's, if architectural) checklist before merge.

3. Commit Messages

Conventional, imperative-mood, referencing the FR/BR/US/Bug ID (Ch.7.6) — e.g. 'Add checklist retry action (FR-CHK-05, C-08)' or 'Fix upload retry loop (BUG-014).' An AI-authored commit follows this exact same format; there is no separate convention for AI-generated commits.

4. Pre-Commit / Pre-Push

Chapter 7.6's git hooks run static analysis locally before a commit lands — an AI session does not bypass or disable these hooks to get a commit through faster.

5. What Never Gets Committed

- .env files, service-account keys, or any secret (Ch.7.6/7.10) — an AI session double-checks a diff for accidental secret inclusion before proposing a commit, since this is one of the few mistakes that's costly to reverse after a push.

6. Release Tagging

A merge to main that reaches a Changelog-worthy version (Volume 11, Ch.11.5) is tagged with the same versionName (Volume 10, Ch.10.1) — the git tag, the changelog entry, and the store release all reference the identical version string.

VOLUME 12 — AI DEVELOPMENT HANDBOOK

CHAPTER 12.10

Definition of Done

The Single Authoritative Standard Every Story Is Held To

Document Title	Definition of Done
Chapter Reference	12.10
Version	1.0
Status	Draft — Pending Approval
Owner	Faisal — Human Archive
Prepared With	Claude (Cowork)
Date	05 August 2026

1. Why This Is Here, Not in Volume 11

Volume 11, Chapter 11.3 deliberately deferred this definition to avoid a second, competing version existing anywhere else. This is that single authoritative definition — every US-*, FR-*, and bug fix is measured against exactly this list, nowhere else.

2. The Definition

A story/change is Done only when every item below is true — no partial credit:

- Implements the exact FR/BR/US it targets (Volume 1) — not a close approximation.
- Passes Chapter 12.5's Code Review Checklist (or 12.6's, if architectural).
- Passes Chapter 12.7's Testing Checklist, including coverage targets (Volume 9, Ch.9.5).
- Static analysis clean, zero suppressed warnings (Volume 3, Ch.3.7).
- Documentation updated per Chapter 12.4 if the change alters described behavior.
- Feature Tracker (Volume 11, Ch.11.6) status updated.
- Changelog (Volume 11, Ch.11.5) entry added if user-facing.
- Bug Tracker (Volume 11, Ch.11.7) entry closed, if this change was a bug fix.
- Merged to main via the Chapter 12.9 workflow — not left on a long-lived branch.

3. Milestone-Level Done

A Milestone (Volume 11, Ch.11.2) is Done only when every story feeding it is Done by this definition — a milestone is never marked complete on 'mostly done' stories.

4. Enforcement

This checklist is applied the same way regardless of whether the work was written by Faisal or generated by an AI session — Chapter 12.1's core principle (no lower bar for AI-generated work) applies at the finish line as much as at the start.